

Takarabako (宝箱) — Product Requirements Document

Event: ETHGlobal Tokyo 2026 (Sept 25–27, 36-hour hackathon, Toranomon Hills)

One-liner: A physical IoT deposit box that turns Japanese cash into on-chain yen yield — insert cash, verify you're human, earn yield on JPYC in an ERC-4626 vault.

Status: Draft v0.1 — hackathon scope

Owner: Wanaqil

1. Problem

Japan remains a cash-heavy economy; physical yen circulates widely and sits idle. There is no simple, trusted way for an ordinary person holding cash to put that cash to work on-chain — every existing path requires a bank account, an exchange KYC flow, a smartphone wallet seed phrase, and a tolerance for crypto UX. The on-ramp itself is the wall.

Takarabako collapses the on-ramp into a single physical action: **insert a banknote, walk away earning**

yield. The blockchain is invisible; the box is the interface.

2. Target users

- **Primary (demo persona):** A Japanese resident comfortable with cash but not with crypto, who wants a savings/yield product without opening an exchange account.
- **Secondary:** Tourists holding leftover yen who want to park it productively rather than convert at a bad airport rate.
- **Operator persona:** Whoever runs the box — they fund the treasury, bear the float, and handle compliance. (See §10.)

3. Goals & non-goals (36-hour scope)

Goals

- A working physical loop: cash inserted → on-chain JPYC deposit → vault position → visible yield.
- Proof-of-personhood gating via World ID (one verified human → one account, sybil-resistant).
- An ERC-4626 vault holding JPYC that accrues yield and supports deposit/withdraw.

- A live, demoable screen on the box showing balance + accruing yield.
- Clean sponsor-track alignment (World, Uniswap, JPYC).

Non-goals (explicitly out of scope for the hackathon)

- Cash **dispensing** (note dispensers are expensive, fragile hardware – withdrawal is crypto-out only in v1).
- Real fiat custody / regulatory licensing (faked with a faucet-funded treasury on testnet).
- Production AML/KYC (stubbed; see §10).
- Mainnet deployment with real JPYC liquidity (testnet + mock JPYC; see §7.5).

4. Definition of done (the demo)

The judge inserts a banknote (or presses the fallback button). Within ~10 seconds the box screen shows: *"¥1,000 deposited – earning 4.2% APY – balance ¥1,003 and rising."* The judge scans a World ID QR to claim/withdraw, and the JPYC lands in their wallet. That full loop, live, is the win.

5. System architecture

Five layers, cleanly separated:

Layer	Responsibility	Tech
Physical	Accept cash, read denomination, drive screen	Raspberry Pi + bill acceptor module (+ button fallback)
Device agent	Read acceptor signal, talk to backend, render UI	Python/Node on the Pi, local web UI
Backend / orchestrator	Hold treasury key, front JPYC, sign txns, expose API	Node/TypeScript service + ethers/viem
Identity	Proof-of-personhood, account binding	World ID (IDKit + on-chain verify)
On-chain	Settlement + yield	JPYC (ERC-20) + Takarabako ERC-4626 vault (+ optional Uniswap v4 strategy)

Architectural truth to internalize

When cash enters the box, the **cash sits physically in the machine** — the JPYC credited to the user is fronted from a pre-funded **treasury hot wallet**.

Takarabako is therefore not "converting" cash; it is running a float against incoming cash. This is the real business (and the real regulatory surface). For the hackathon the treasury is a testnet faucet wallet.

Name this openly to judges as the v2 path — don't let them surprise you with it.

6. Core user flows

6.1 Deposit (cash → yield)

1. User inserts banknote.
2. Bill acceptor emits denomination signal → device agent reads it.
3. Device agent requests/confirms World ID session (QR on screen).
4. Device agent → backend: `POST /deposit` with `{ amount, nullifierHash }`.
5. Backend validates the World ID proof + nullifier, then calls `vault.depositFor(user, amount)` funded from treasury.
6. Vault mints shares to the user's bound address; screen confirms balance + APY.

6.2 Withdraw (yield → crypto out)

1. User scans World ID QR → proves they are the same human (same nullifier).
2. Backend calls `vault.withdrawTo(user, shares)`.
3. JPYC (principal + accrued yield) transfers to the user's wallet. No cash leaves the box.

7. Component specs

7.1 Hardware

- **Raspberry Pi 4/5** (or Arduino + Pi; Pi alone is simpler for networking).
- **Bill acceptor module** (e.g. an NV9/ICT/Pyramid-style validator) outputting **pulse or ccTalk/serial** per accepted note.
- **Connectivity**: Wi-Fi (primary) with a SIM/4G dongle as backup for the venue.
- **Display**: small HDMI/touch screen or the Pi driving a local browser kiosk page.
- **⚠ Fallback (build this FIRST)**: a physical button or on-screen control where 1 press = ¥1,000. The bill acceptor protocol (pulse vs ccTalk) is the single biggest time sink at a 36-hour event. The entire on-chain loop must demo perfectly *without*

the validator, then you bolt the validator on if time allows.

7.2 Device agent

- Reads acceptor (GPIO interrupt for pulse, or pyserial for ccTalk).
- Maps pulse count / serial code → denomination.
- Renders kiosk UI (balance, APY, QR for World ID).
- Calls backend over HTTPS; holds **no private keys**.

7.3 Backend / orchestrator

- Stateless API + a single treasury signer (env-injected key, testnet only).
- Endpoints:
 - `POST /session` → returns World ID action params + QR payload.
 - `POST /deposit` → `{ amount, worldIdProof }` → verifies, calls `depositFor`, returns tx hash.
 - `POST /withdraw` → `{ worldIdProof }` → resolves user, calls `withdrawTo`.
 - `GET /position/:user` → `{ shares, jpycValue, apy }` for the screen.
- Verifies World ID proof server-side before any treasury spend.

7.4 Smart contracts

Takarabako Vault (ERC-4626 over JPYC):

```
interface ITakarabakoVault {
    // Treasury fronts `jpycAmount`; shares
    minted to `user`.
    function depositFor(address user,
        uint256 jpycAmount) external returns
        (uint256 shares);

    // Burns shares, sends principal +
    yield in JPYC to `user`.
    function withdrawTo(address user,
        uint256 shares) external returns (uint256
        jpycAmount);

    function sharesOf(address user)
    external view returns (uint256);
    function previewValue(address user)
    external view returns (uint256 jpycValue);
    // incl. yield
    function currentApyBps() external view
    returns (uint256);
}
```

Identity gate (World ID verification, called once per user to bind an address):

```
function registerUser(
    address userAddress,      // signal:
    the address being bound
    uint256 root,
```

```
uint256 nullifierHash,    // one per
human → prevents double-registration
uint256[8] calldata proof
) external; // reverts if proof invalid or
nullifier already used
```

7.5 Settlement asset — JPYC

- **JPYC** is Japan's first FSA-licensed yen stablecoin (live since Oct 2025, on Ethereum / Avalanche / Polygon) — a real composable ERC-20.
- **Hackathon reality:** deploy a **MockJPYC (ERC-20)** on a testnet (Sepolia / Base Sepolia) for the demo, with a comment that mainnet swaps in the real JPYC address. Do not block the demo on real JPYC testnet liquidity.
- *Skip JPYSC* (SBI/Shinsei trust-bank coin) — it launched June 24 2026 but is gated to SBI VC Trade account holders, so it is not composable for you.

7.6 Yield strategy

- **Baseline (must work):** a **mock yield vault** — the vault's share/asset exchange rate increases over time at a fixed APR, topped up from a rewards wallet. This makes yield *visibly tick up* in the demo without depending on a live integration. Label it honestly as simulated.

- **Stretch (prize-aligned):** route the vault's JPYC through a **Uniswap v4 pool / hook** as the real strategy, or a lending market position. Only attempt after the baseline loop is green.

8. Sequence diagram

```
sequenceDiagram
    actor User
    participant Box as Bill Acceptor / Pi
    participant Agent as Device Agent
    participant API as Backend (Treasury)
    participant WID as World ID
    participant Vault as ERC-4626 Vault (JPYC)

    User->>Box: Insert ¥1,000 (or press fallback)
    Box->>Agent: denomination signal
    Agent->>User: show World ID QR
    User->>WID: scan & prove personhood
    WID-->>Agent: proof + nullifierHash
    Agent->>API: POST /deposit {amount, proof}
    API->>WID: verify proof (server-side)
    WID-->>API: valid
    API->>Vault: depositFor(user, amount)
    // funded from treasury
    Vault-->>API: shares minted (tx hash)
    API-->>Agent: 200 {balance, apy}
    Agent->>User: "¥1,000 deposited – 4.2% APY"
```

```
Note over Vault: exchange rate accrues
over time
User->>Agent: scan QR to withdraw
Agent->>API: POST /withdraw {proof}
API->>Vault: withdrawTo(user, shares)
Vault-->>User: JPYC (principal + yield)
```

9. Data model

- **User:** { worldIdNullifier (PK), boundAddress, createdAt }
- **Position:** derived on-chain (sharesOf , previewValue); backend caches for the screen.
- **DepositEvent:** { id, nullifier, denomination, txHash, ts } (for the demo dashboard / receipts).

10. Security & compliance notes

- **Treasury key:** lives only in the backend, testnet only, env-injected, never on the Pi. The Pi is a dumb terminal.
- **Float / custody:** the operator bears the cash-vs-crypto float. In production this is a registered money-transfer / exchange activity under Japan's Payment Services Act — JPYC itself operates AML screening (Elliptic) under a fund-transfer license with a ~¥1M cap. Out of scope to *implement*, in scope to *articulate*.

- **AML stub:** mention an Elliptic-style screening hook on withdrawal as the v2 compliance path.
- **Sybil resistance:** World ID nullifier ensures one human = one account; reused nullifiers revert.

11. Sponsor / prize mapping

Confirm against the official Tokyo 2026 sponsor list when it drops; ETHGlobal limits projects to **3 sponsor SDKs**. The targets below fit exactly three.

Track (target)	How Takarabako uses it	Why it fits
World / World ID	Proof-of-personhood gate; one human → one account	Sybil resistance + clean identity story; World has sponsored the 2026 ETHGlobal circuit
Uniswap Foundation	v4 hook / pool as the yield strategy (stretch)	v4-hooks track appeared at ETHGlobal NYC 2026
JPYC	Native yen settlement	Hyper-relevant, regulator-aware, Japan-local —

Track (target)	How Takarabako uses it	Why it fits
	asset for the whole loop	strong judge appeal
<i>Optional:</i> OG / others	Off-chain receipts or data availability	Only if a 3rd slot is free and a clean fit exists

12. 36-hour build plan

Phase 0 — setup (hrs 0–3): repo, testnet wallets, deploy MockJPYC, scaffold backend + Pi kiosk page.

Get the fallback button → on-chain deposit working end-to-end before touching hardware.

Phase 1 — vault + loop (hrs 3–12): ship ERC-4626 vault with mock yield; wire `depositFor` / `withdrawTo`; backend treasury signing; screen shows balance + ticking APY. *Milestone: full software loop green.*

Phase 2 — identity (hrs 12–20): integrate World ID (IDKit QR → server verify → bind address). Gate deposit/withdraw on a valid nullifier.

Phase 3 — hardware (hrs 20–30): interface the real bill acceptor (pulse first; ccTalk only if time). Keep the button fallback wired in parallel as the safety net.

Phase 4 — polish + stretch (hrs 30–36): demo UI polish, receipts dashboard, optional Uniswap v4 strategy swap-in, rehearse the demo script twice.

13. Risks & mitigations

Risk	Likelihood	Mitigation
Bill acceptor protocol eats the weekend	High	Button fallback built first; validator is additive
World ID verify integration friction	Medium	Stub a "verified" mode; swap real verify in once loop works
No JPYC testnet liquidity	Medium	MockJPYC ERC-20; note mainnet address swap
Float/custody question from judges	High	Pre-script the answer; frame as named v2 path
Live yield integration breaks demo	Medium	Mock-yield baseline; real strategy is stretch only

14. Demo script (90 seconds)

1. "Japan is cash-first. Crypto on-ramps are a wall. Watch this." → insert note.
2. Screen: "¥1,000 deposited – 4.2% APY." → "Cash just became on-chain yen yield. No exchange, no seed phrase."
3. Scan World ID QR → "One human, one account, sybil-proof – via World ID."
4. Point at ticking balance → "Earning live in an ERC-4626 JPYC vault."
5. Withdraw → JPYC hits wallet. "Here's the honest part: the box holds the cash; the JPYC is fronted from a treasury – which in production is the regulated money-transfer layer. That's our roadmap."

15. Future (v2+)

- Cash-out via note dispenser.
- Real fund-transfer license + AML pipeline + treasury reconciliation.
- Multi-box fleet with shared treasury and yield routing.
- Real Uniswap v4 / lending strategies; JPYC mainnet.

- Tourist mode (FX-aware) and receipt NFTs.